Phase 2 - Week 6 - Lesson Day 2

Random Forests

Today's Goal

Yesterday we saw that one decision tree can overfit. Today's fix is simple: instead of trusting one tree, build **many** trees and let them vote. That is a **Random Forest**.

- Understand why many trees beat one tree
- Train a Random Forest with Scikit-learn
- See which inputs the model finds most important

1. What Is a Random Forest?

> **Simple definition:** A random forest is a group of many decision trees. Each tree sees a slightly different random piece of the data, so each tree learns slightly different questions. To make a prediction, every tree votes, and the answer with the most votes wins.

Think of asking 100 different friends for advice instead of just one. One friend might be wrong, but the majority is usually right. Different mistakes from different trees cancel each other out.

This idea (combining many models into one) is called an **ensemble**. You will see it again later with XGBoost.

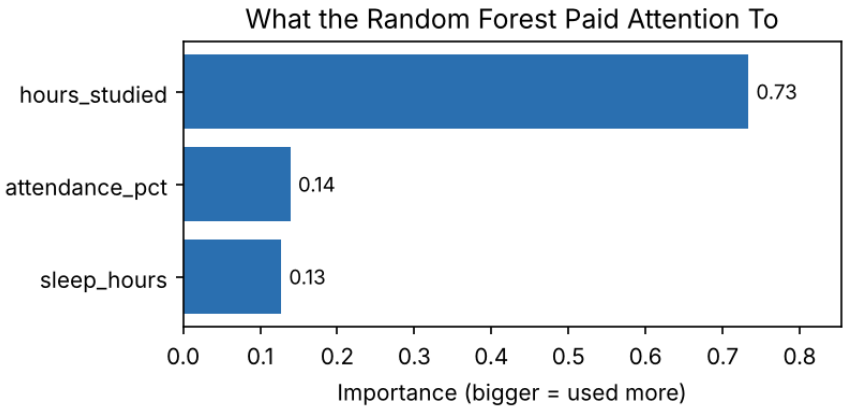
2. Training a Random Forest

```
from sklearn.ensemble import RandomForestClassifier

forest = RandomForestClassifier(n_estimators=100, random_state=42)
forest.fit(X_train, y_train)

print(forest.score(X_test, y_test))   # accuracy on the test data
```

n_estimators is the number of trees in the forest. 100 is a good starting number. More trees are usually a little better and a little slower.

3. Which Inputs Matter Most?

Simple definition: A forest can tell you how much it relied on each input column. This is called **feature importance**. The numbers add up to 1, and a bigger number means that column was more useful for predicting.



In our student data, **hours_studied** is by far the most important (about 0.73). **attendance_pct** and **sleep_hours** are both small and close together (about 0.13 to 0.14). That fits how we built the data: hours has a big effect on the score, attendance only a small effect, and sleep no effect at all.

One careful note: sleep got a score above zero even though it does nothing. A forest gives every column a little importance, because it finds small accidental patterns in noise. So read feature importance as a rough guide ("hours is the big one"), not as exact truth.

```
importance = pd.Series(forest.feature_importances_, index=X.columns)
print(importance.sort_values(ascending=False))
```

Why this is useful at work: it tells you which columns to focus on, and which columns you could remove to make a simpler model.

4. Is a Forest Always Better?

Usually a forest is steadier and less overfit than one tree. But it is not magic. On simple data it may score about the same as a single tree. You always check by comparing test scores, never by assuming.

Today's Practice

Open **day-2-random-forests.ipynb**. All answers are already solved.

Practice Questions (5 total - all solved)

1. Train a forest

Train a RandomForestClassifier with 100 trees.

2. Forest vs one tree

Compare the test score of the forest to a single tree with max_depth=3.

3. Feature importance

Print the importance of each input column, from biggest to smallest.

4. How many trees?

Train forests with 5, 50, and 300 trees and compare their test scores.

5. Predict a new student

Predict pass/fail and the probability for a new student who studied 6 hours, with 80 percent attendance and 7 hours of sleep.

Once you and Halimo finish, move to Lesson Day 3 (Evaluation Metrics).